The Vocabulary Problem in Human-System Communication

中英對照・雙核心複核版｜人機系統溝通中的詞彙問題

G. W. Furnas ・ T. K. Landauer ・ L. M. Gomez ・ S. T. Dumais
Communications of the ACM, 30(11), 964-971 ・ 1987
DOI: 10.1145/32206.32212

逐段中英對照｜保留原始頁面、圖表與圖說｜另附口語研讀摘要

閱讀與複核說明

每一頁先完整保留英文原始 PDF 頁面，後接該頁經 Claude
逐欄、逐表核對的繁體中文譯文。原文影像是英文正本，中文為研讀輔助。

本版已放棄容易造成雙欄錯序的自動逐段 OCR 翻譯，改以原始頁面影像與人工結構化中文逐頁對照。

表格數值與 OCR 爭議均以 400–500 dpi 原始頁面影像為準；Results Table III 的 Common Objects 為 .28，RAISIN 頻次為 1。

現代語意搜尋、向量資料庫與 RAG
的連結屬本次研讀延伸，不是作者原始主張。正式引用時請回查原文印刷頁碼 964–971。

複核分工：Codex 建檔、排版與驗證；Claude
第二核心完成全文語意、雙欄順序、圖表、數據與書目交叉複核。

RESEARCH CONTRIBUTIONS

Human Aspects of Computing

Henry Ledgard Editor

The Vocabulary Problem in Human–System Communication

G. W. FURNAS, T. K. LANDAUER, L. M. GOMEZ, and S. T. DUMAIS

ABSTRACT: *In almost all computer applications, users must enter correct words for the desired objects or actions. For success without extensive training, or in first-tries for new targets, the system must recognize terms that will be chosen spontaneously. We studied spontaneous word choice for objects in five application-related domains, and found the variability to be surprisingly large. In every case two people favored the same term with probability <0.20. Simulations show how this fundamental property of language limits the success of various design methodologies for vocabulary-driven interaction. For example, the popular approach in which access is via one designer's favorite single word will result in 80–90 percent failure rates in many common situations. An optimal strategy, unlimited aliasing, is derived and shown to be capable of several-fold improvements.*

1. INTRODUCTION

Many functions of most large systems depend on users typing in the right words. New or intermittent users often use the wrong words and fail to get the actions or information they want. This is the vocabulary problem. It is a troublesome impediment in computer interactions both simple (file access and command entry) and complex (database query and natural language dialog).

In what follows we report evidence on the extent of the vocabulary problem, and propose both a diagnosis and a cure. The fundamental observation is that people use a surprisingly great variety of words to refer to the same thing. In fact, the data show that no single access word, however well chosen, can be expected to cover more than a small proportion of users' attempts. Designers have almost always underestimated the problem, and, by assigning far too few alternate entries to databases or services, created an unnecessary barrier to effective use. Simulations and direct experimental tests of several alternative solutions show that rich, probabilistically weighted indexes or alias lists can improve success rates by factors of three to five.

A much longer, highly technical version of some early parts of this work appeared in [3].

2. THE VOCABULARY PROBLEM AS VARIABILITY IN WORD USAGE

If everyone always agreed on what to call things, the user's word would be the designer's word would be the system's word, and what the user typed or pointed to would be mutually understood. Unfortunately, people often disagree on the words they use for things. To experience the problem, try the exercise in Figure 1. It is given here in the context of a command access ("command naming") problem, but it could just as well have come from library information retrieval or database query. Ask one other colleague to also try it and compare your responses.

We have asked this of more than a thousand pairs of people, including programmers, computer science students, and human factors specialists. Less than a dozen pairs have agreed. This means that without tutoring, less than a dozen people out of a thousand could have directly accessed the command, had their partner

On a piece of paper write the name you would give to a program that tells about interesting activities occurring in some major metropolitan area (e.g., this program would tell you what is interesting to do on Friday or Saturday night). Make the name 10 characters or less.

Try to think of a name that will be as obvious as possible, one that other people would think of.

FIGURE 1. To Demonstrate How Rarely Two People Will Agree on What to Call Things, Ask One Other Colleague to Try it too and Compare Your Responses

印刷頁碼 964｜繁體中文譯文

版面標註（不得併入標題）

〔期刊專欄報頭〕Human Aspects of Computing（電腦運算的人因面向）｜專欄編輯：Henry Ledgard

標題

人機系統溝通中的詞彙問題（The Vocabulary Problem in Human–System Communication）

作者

G. W. Furnas、T. K. Landauer、L. M. Gomez、S. T. Dumais

ABSTRACT（摘要）

在幾乎所有的電腦應用中，使用者都必須輸入正確的詞，才能取得想要的物件或動作。若要在未經大量訓練的情況下成功，或在初次嘗試新目標時就成功，系統就必須能辨識使用者會自發選用的詞。我們研究了五個與應用相關領域中，人們為項目自發選詞的情形，發現其變異性大得驚人。在每一個案例中，兩個人偏好同一個用語的機率都低於 0.20。模擬結果顯示，語言的這項基本性質如何限制了各種詞彙驅動互動設計方法的成效。舉例來說，有一種常見做法是只透過某位設計者最偏好的單一詞來存取，這在許多常見情境中會造成 80–90% 的失敗率。本文推導出一種最佳策略——不限量別名（unlimited aliasing）——並顯示它能帶來數倍的改善。

1. INTRODUCTION → 1. 前言

大多數大型系統的許多功能，都取決於使用者是否輸入了正確的詞。新手或偶爾使用的使用者常會用錯詞，因而得不到他們想要的動作或資訊。這就是詞彙問題。無論在簡單的電腦互動（檔案存取與命令輸入）或複雜的電腦互動（資料庫查詢與自然語言對話）中，它都是一項棘手的障礙。

接下來我們將提出詞彙問題嚴重程度的證據，並同時提出診斷與對策。最根本的觀察是：人們用來指稱同一件事物的詞，其種類多得出乎意料。事實上，資料顯示無論選得多好，任何單一存取詞所能涵蓋的使用者嘗試都不會超過一小部分。設計者幾乎總是低估了這個問題；他們替資料庫或服務指派的替代詞條目遠遠太少，因而為有效使用製造了不必要的障礙。針對數種替代解法所做的模擬與直接實驗檢驗顯示，內容豐富、依機率加權的索引或別名清單，能將成功率提高三到五倍。

未標號註腳

本研究部分早期內容有一個篇幅長得多、技術性也高得多的版本，發表於 [3]。

版權列（不譯，照錄）

2. THE VOCABULARY PROBLEM AS VARIABILITY IN WORD USAGE → 2. 詞彙問題即用詞的變異性

如果每個人對事物的稱呼始終一致，那麼使用者的詞就會是設計者的詞，也就是系統的詞；使用者所輸入或指向的東西便能被彼此理解。可惜的是，人們對事物的用詞經常不一致。想親身體驗這個問題，可以試試圖 1 的練習。這裡是以命令存取（「命令命名」）問題的脈絡呈現，但它同樣可能出自圖書館資訊檢索或資料庫查詢。請另一位同事也做一次，然後比對彼此的答案。

我們已經對超過一千對受試者做過這個練習，包括程式設計師、資訊科學學生與人因專家。其中答案一致的不到十幾對。這表示在沒有教學的情況下，若由夥伴替該命令命名，一千個人裡不到十幾個人能直接存

取到它。

Figure 1 方框內文（練習指示）

請在紙上寫下你會替一個程式取的名稱；這個程式會告訴你某個大都會區裡正在發生的有趣活動（例如，它會告訴你週五或週六晚上有什麼好玩的事可做）。名稱請控制在 10 個字元以內。

請盡量想一個愈明顯愈好的名稱，也就是別人也會想到的名稱。

FIGURE 1 圖說

圖 1. 為了證明兩個人對事物的稱呼有多難得一致，請另一位同事也試做一次，並比對你們的答案。

named it. (As a follow-up exercise, list other good names you think of. You will be able to compare your list against one we reveal later.)

In current computer systems, the vocabulary problem is largely ignored. Designers decide on the terms to be used, and, as heavy users, grow to find these terms obvious and natural. Other users are simply required to learn the system's words.[1]

In information retrieval systems, the keywords that are assigned by indexers are often at odds with those tried by searchers. The seriousness of the problem is indicated by the need for professional intermediaries between users and systems, and by disappointingly low average performance (recall) rates.

The standard conceptualization of the naming problem is to begin with the system's objects or functions, and ask what word or words should be associated with each: What name for this command? What keyword for this document? In Section 3.5, we will argue that this approach is basically misleading, but will explore it first. Throughout we take an empirical approach, collecting large amounts of data on actual human language usage, then modeling and evaluating different system strategies. A more detailed exposition of the data collection and analysis methods can be found in [4].

3. THE DATA

In an attempt to identify some basic principles of broad generality, we collected data on five very different but typical application domains. The words in square brackets are titles for the data sets, and will be used in the tables that follow.

(1) Main verbs applied by 48 typists to describe manual text editing operations, for 5 generic operations or 25 combinations of operations and text units [Editor-5, Editor-25]. The typists saw author-marked corrections and provided brief instructions as if to another typist.

(2) Commands for a hypothetical "message decoder" program ["Decoder"], nominated by 100 experienced system designers after studying the design and interface.[2]

(3) The first content word used in describing a selection of 50 common objects ["Common objects"], given by 337 college students. They were shown short verbal descriptions such as "love," "motorcycle," or "Newsweek," and asked to give alternative words or phrases for a person or computer to understand.

(4) First-nominated superordinate category for 64 "Swap 'n Sale" classified ad items ["Classifieds"], given by 30 New Jersey homemakers.

(5) First priority keyword nominated for a computerized file of 188 recipes. ["Recipe Keywords"], by 8 expert cooks and 16 homemakers.

These domains all involved information objects that one might want to access on a computer and were all of modest size (5–200 objects), but in most other respects they differed: in knowledge domain, in elicitation technique, in type of people, in preprocessing (e.g., ignoring endings or noncontent words). Note also that the first two relate to command access, the third to general knowledge description, and the last two to information retrieval. Conclusions that apply to all of these disparate domains must be based on general rather than idiosyncratic properties, and are therefore likely to hold for many others.

In what follows we will use "object" or "item" to refer to any of the entities (text editor operations, decoder commands, common objects, advertised items, and cooking recipes) which we presented to people as stimuli. The words we got back we will call "terms," "words," "descriptors," or "names."

3.1 Analyses

For each domain, we counted how often each word was applied to each object. Samples of the resulting data are shown in Tables Ia and Ib. (Objects are denoted at the top of each column by abbreviated examples or descriptions: the actual 'objects' seen by the subjects, as summarized above, are given in [4].)

For example Table Ia indicates that 22 subjects proposed the word "change" as a descriptor for the editing operation indicated by a crossed out word by an author, and referred to in the table as "*delete*."

All the tables were very sparse. One reason is that word usage tended to follow Zipf's distribution [14]—a few words are used very frequently, the vast majority only rarely; more importantly however, most words are applied to only a few objects.

Because they estimate the likelihood of users or designers giving a word for an object, these tables allow us to simulate the performance of various approaches to word-based access. To see how untutored users' words would fare against a system, we can simply sample from these tables instead of going back to subjects.

3.2 "Armchair" Naming

The simplest version of vocabulary-based access is when objects are accessible through a single special term—its "name." The most common strategy is for designers to install their own personal favorites as object names. We call this the "Armchair" design method. It gives users access only if their word coincides with the designer's.

For our data sets the probabilities that two people (e.g., a user and designer) coincide on the word they spontaneously apply to a given object is presented in Results Table I.[3]

[1] Landauer, Galotti, and Hartwell [8] found that the initial learning of an editor by beginners was not much affected by the differences in the "naturalness" of the small number of terms they needed to learn. However, learning a few names is not the major problem in mastering one's first computer application while knowing the right terms may be a very large factor in the effective use of extensive systems.

[2] Data from P. Barnard, Applied Psychology Unit, Cambridge, England, personal communication.

[3] We use an unbiased statistical estimator of this quantity, the *repeat rate* given in [7].

印刷頁碼 965｜繁體中文譯文

左欄（承前）

……替它命名的話。（作為後續練習，請再列出你想到的其他好名稱，稍後可以和我們公布的清單比對。）

在目前的電腦系統中，詞彙問題大多被忽視。設計者決定要使用的詞，而由於他們是重度使用者，久而久之便覺得這些詞理所當然、自然而然。其他使用者則只能被要求去學系統的用詞。[1]

在資訊檢索系統中，索引員指派的關鍵詞經常與檢索者嘗試的關鍵詞對不上。這個問題的嚴重性，可以從使用者與系統之間需要專業中介者、以及平均績效（召回率）低得令人失望看出來。

命名問題的標準思考方式，是從系統的物件或功能出發，再問應該替每一個配上哪一個或哪些詞：這個命令該叫什麼名稱？這份文件該用什麼關鍵詞？在 3.5 節我們將論證這種取徑根本上具有誤導性，但會先沿著它探討。全文我們採取實證取徑：先蒐集大量人類實際語言使用的資料，再對不同的系統策略進行建模與評估。資料蒐集與分析方法的更詳細說明見 [4]。

3. THE DATA → 3. 資料

為了找出一些具有廣泛普遍性的基本原則，我們蒐集了五個差異極大但具代表性的應用領域的資料。方括號中的詞是各資料集的名稱，後續表格會使用。

(1) 48 位打字員用來描述人工文字編輯操作的主要動詞，涵蓋 5 種通用操作，或 25 種操作與文字單位的組合〔Editor-5、Editor-25〕。打字員看到作者標註的修改，並像是要交代給另一位打字員那樣寫下簡短指示。

(2) 100 位資深系統設計者在研究過設計與介面後，為一個假想的「訊息解碼器」程式所提名的命令〔"Decoder"〕。[2]

(3) 337 位大學生在描述所選的 50 個常見物件時所使用的第一個實詞〔"Common objects"〕。他們看到的是簡短的文字描述，例如 "love"、"motorcycle" 或 "Newsweek"，並被要求提供可讓他人或電腦理解的替代詞或片語。

(4) 30 位紐澤西州家庭主婦為 64 則 "Swap 'n Sale" 分類廣告項目最先提名的上位類別〔"Classifieds"〕。

(5) 8 位專業廚師與 16 位家庭主婦為一個收錄 188 道食譜的電腦化檔案所提名的第一優先關鍵詞〔"Recipe Keywords"〕。

右欄接續

這些領域都涉及人們可能想在電腦上存取的資訊項目，規模也都不大（5–200 個項目），但在其他多數面向上彼此不同：知識領域不同、誘出（elicitation）技術不同、受試者類型不同、前處理方式也不同（例如是否忽略字尾或非內容詞）。另外請注意，前兩者屬於命令存取，第三者屬於一般知識描述，最後兩者屬於資訊檢索。凡是能適用於這些互異領域的結論，必定建立在一般性而非特異性的性質上，因此很可能也適用於許多其他領域。

接下來，我們會用「物件（object）」或「項目（item）」來指稱我們呈現給受試者作為刺激的任何實體（文字編輯器操作、解碼器命令、常見物件、廣告刊登項目與烹飪食譜）。而我們回收到的詞，則稱為「用語（terms）」、「詞（words）」、「描述詞（descriptors）」或「名稱（names）」。

3.1 Analyses → 3.1 分析

針對每個領域，我們統計了每個詞被套用到每個項目的次數。所得資料的樣本見表 Ia 與表 Ib。（各欄頂端以簡略的例子或描述標示項目；受試者實際看到的「物件」如前所述，完整內容見 [4]。）

例如表 Ia 顯示，有 22 位受試者提出 "change" 一詞，來描述作者用刪除線劃掉某個字所代表的編輯操作，該操作在表中稱為 "delete"。

所有表格都非常稀疏。原因之一是用詞傾向遵循齊夫分布（Zipf's distribution）[14]——少數幾個詞被非常頻繁地使用，絕大多數的詞只偶爾出現；但更重要的是，大多數的詞只被套用到少數幾個項目上。

由於這些表格能估計使用者或設計者為某個項目提供某個詞的可能性，它們讓我們得以模擬各種以詞為基礎的存取方法的表現。想知道未受指導的使用者所用的詞在系統面前會有什麼下場，我們只要從這些表格取樣即可，不必再回頭找受試者。

3.2 "Armchair" Naming → 3.2 「直覺（Armchair）」命名

以詞彙為基礎的存取，最簡單的版本就是每個項目只能透過單一的特殊用語——它的「名稱」——來存取。最常見的策略是設計者把自己的個人偏好裝進系統，當成項目名稱。我們稱這種設計方法為「直覺命名（Armchair）」法。只有當使用者的詞剛好與設計者的詞一致時，使用者才進得去。

在我們的資料集中，兩個人（例如一位使用者與一位設計者）對某個項目自發使用同一個詞的機率，列於結果表 I。[3]

註腳（三則，須分開且不得跨欄黏連）

[1] Landauer、Galotti 與 Hartwell [8] 發現，初學者一開始學習某個編輯器的成效，並不太受到他們必須學會的少數幾個詞「自然程度」差異的影響。不過，學會少數幾個名稱並不是掌握人生第一套電腦應用時的主要難題；而在有效使用大型系統時，知道正確的用詞可能是非常重要的因素。

[2] 資料來自英國劍橋應用心理學單位（Applied Psychology Unit, Cambridge, England）P. Barnard 的私人通訊。

[3] 我們採用此量的不偏統計估計量，即 [7] 所給出的重複率（repeat rate）。

TABLE I. Word-Object Data

(a) *Sample data from the text-editing study*

Words	Objects					
	"Insert"	*"Delete"*	*"Replace"*	*"Move"*	*"Transpose"*	...
Change	30	22	60	30	41	
Remove	0	21	12	17	5	
Spell	4	14	13	12	10	
Reverse	0	0	0	0	27	...
Leave	10	0	0	1	0	
Make into	0	4	0	0	1	
.	.	.	.	.	.	
.	.	.	.	.	.	
.	.	.	.	.	.	

(b) *Sample data from the common object study*

Words	Objects						
	"Calculator"	*"Nectarine"*	*"Lucille Ball"*	*"Pear"*	*"Raisin"*	*"Robin"*	...
Machine	4	0	0	0	0	0	
Green	0	0	0	7	0	0	
Bird	0	0	0	0	0	21	...
Fruit	0	12	0	19	1	0	
Red	0	0	8	0	0	7	
Female	0	0	2	0	0	0	
.	.	.	.	.	.	.	
.	.	.	.	.	.	.	
.	.	.	.	.	.	.	

Results Table I
Probability of two people applying the same term to an object

Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
.07	.11	.08	.12	.14	.18

Clearly people disagree greatly in the labels they use. The probability that two typists will use the same main verb in describing an editing operation is less than one in fourteen; that two cooks will use the same first keyword for a recipe is less than one in five.

The data tell us that the armchair method is highly unsatisfactory. If one person assigns the name of an item, other untutored people will *fail* to access it on 80 to 90 percent of their attempts. This finding is not only true of all six of our laboratory data sets; it has also been confirmed several times by research with actual systems (Furnas [4]; Gomez and Lochbaum [5]; Good, Whiteside, Wixon, and Jones [6]).

One might be tempted to think that domain experts could do a better naming job. In the recipe study, one third of keyword providers were expert cooks. We found, however, that their keywords fared no better than average, either in indexing for other experts or for novices. Similarly the decoder command names were chosen by and for experts, and had very low agreement.

So far we have assumed no requirement that access terms be distinct, that is, that each name be assigned to only one object. While in information retrieval contexts, several documents may share a keyword, for command or file names uniqueness is typically important: the system must have an unambiguous interpretation for the intended action and objects. Adding such a constraint obviously means that good candidate names for one object often get ruled out, having already been assigned to another object. As a result, untutored people will be even less likely to hit upon the official name.

In our data, requiring uniqueness caused proportional decrements of 5 to 60 percent (typically about 10 percent) in the already low performances. Constraining names to be multiple word strings, systematic, "cute," or even "mnemonic," is certain to result in even less likely first-try success. For example Good, et al. [6] found that their system's initial, armchair-named commands, were slightly below our predictions, presumably reflecting other constraints imposed by the system design on the name choices.

3.3 How Good is the Best Possible Name?

From the standpoint of first-try success for the untrained, the best possible access term would be the word real users most often apply to an object. This empirically based naming approach has been a popular proposal in human factors circles: terms offered by a representative sample of potential users would be collected, and the most frequent term identified.

We can identify this "best" name from data tables like Table 1. For statistical reasons[4] data can only pro-

[4] There is no way to get an unbiased, distribution-free estimate of the "true" value of the maximum-frequency cell.

印刷頁碼 966｜繁體中文譯文

TABLE I 標題與子標題

表 I. 詞—項目資料（Word-Object Data）

(a) 文字編輯研究的樣本資料（Sample data from the text-editing study）

(b) 常見物件研究的樣本資料（Sample data from the common object study）

表 Ia（欄＝項目，列＝詞；數值為次數）

Words	"Insert"	"Delete"	"Replace"	"Move"	"Transpose"	…
Change	30	22	60	30	41	
Remove	0	21	12	17	5	
Spell	4	14	13	12	10	
Reverse	0	0	0	0	27	…
Leave	10	0	0	1	0	
Make into	0	4	0	0	1	

表 Ib

Words	"Calculator"	"Nectarine"	"Lucille Ball"	"Pear"	"Raisin"	"Robin"	…
Machine	4	0	0	0	0	0	
Green	0	0	0	7	0	0	
Bird	0	0	0	0	0	21	…
Fruit	0	12	0	19	1	0	
Red	0	0	8	0	0	7	
Female	0	0	2	0	0	0	

Results Table I

結果表 I：兩個人對同一項目使用相同用語的機率

Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
.07	.11	.08	.12	.14	.18

正文

顯然人們在所使用的標籤上分歧極大。兩位打字員在描述同一項編輯操作時使用同一個主要動詞的機率，不到十四分之一；兩位廚師為同一道食譜給出相同的第一個關鍵詞的機率，不到五分之一。

資料告訴我們，直覺命名法非常不理想。如果由某一個人替某個項目命名，其他未受指導的人有 80% 到 90% 的嘗試會存取失敗。這個發現不只適用於我們全部六個實驗室資料集，也已被多項針對真實系統的研究反覆證實（Furnas [4]；Gomez 與 Lochbaum [5]；Good、Whiteside、Wixon 與 Jones [6]）。

或許有人會認為，領域專家能把命名工作做得更好。在食譜研究中，三分之一的關鍵詞提供者是專業廚師。然而我們發現，他們的關鍵詞並沒有比平均水準更好——無論是為其他專家索引，或為新手索引皆然。同樣地，解碼器的命令名稱是由專家為專家所選，一致程度也非常低。

到目前為止，我們都假設存取詞不必互異，也就是說並未要求每個名稱只能指派給一個項目。在資訊檢索的情境中，多份文件可以共用一個關鍵詞；但對命令名稱或檔名而言，唯一性通常很重要：系統必須對意圖的動作與對象有明確無歧義的解讀。加上這樣的限制，顯然會使某個項目的好候選名稱經常因為已經指派給另一個項目而遭到排除。結果是，未受指導的人猜中官方名稱的機會又更低了。

在我們的資料中，要求唯一性會使本來就很低的表現再依比例下降 5% 到 60%（典型約 10%）。若再把名稱限制成多字詞串、系統化、「俏皮」甚至「助記」形式，初次嘗試成功的機會勢必更低。例如 Good 等人 [6] 發現，他們系統最初以直覺命名的命令，表現略低於我們的預測值，推測是系統設計對名稱選擇施加了其他限制所致。

3.3 How Good is the Best Possible Name? → 3.3 最好的名稱能有多好？

從未受訓練者初次嘗試即成功的角度來看，最好的存取詞應該是真實使用者最常套用到該項目的那個詞。這種以實證為基礎的命名取徑，一直是人因領域頗受歡迎的提案：先蒐集具代表性的潛在使用者樣本所提供的用語，再找出最高頻的那個詞。

我們可以從表 I 這類資料表中辨識出這個「最佳」名稱。基於統計上的理由[4]，資料只能提供此方法成功率上下界的估計值。

註 4

[4] 我們無法取得最高頻儲存格「真值」的不偏、且不依賴分布假設的估計。

vide estimates of bounds on the success rate of this method. A low estimate can be obtained by using a random half of the data to pick the best words, and the remaining half to test their effectiveness. A high estimate can be obtained by using the actual relative frequency in the data of the most popular word.

Results Table 2 presents the range of these low and high estimate hit rates for the data sets.

Results Table II
Probability of someone using the most popular term for an object

Editor-5	Editor-25	Decoder	Common Objects	Class-ifieds	Recipe Key-words	
.15	.19	.22	.26	.26	.31	(low estimate)
.16	.22	.21	.28	.34	.36	(high estimate)

While there is typically about a 2 to 1 improvement over the straight armchair method, failures still occur 65–85 percent of the time.

Since no term will be hit upon more often than the most popular term, these numbers represent a true performance limit when a single access term is assigned to each object. Simply stated, the data tell us *there is no one good access term for most objects.* The idea of an "obvious," "self-evident," or "natural" term is a myth! Since even the best possible name is not very useful, it follows that *there can exist no rules, guidelines or procedures for choosing a good name, in the sense of "accessible to the unfamiliar user."*

Any further improvement will require a different strategy.

3.4 The Value of a Few Aliases

Once the idea of a single access term is rejected, the next logical suggestion is to try several access terms. Library catalogues and indices, for example, usually have at least two (but seldom more than four) entry points for each referenced object.

Below we present the estimated performance of a system where each object had three access terms known to the system, selected by frequency-weighted random sampling, mimicking the frequencies of a design group's first three "armchair nominations."

Results Table III
Probability of someone using one of three armchair aliases for an object

Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
.21	.30	.20	.28	.34	.45

While the improvement over a single armchair name is considerable, three armchair aliases are no better than a single optimally chosen term.

If the three most popular, i.e., three optimally chosen access terms, are used, the following results are obtained.

Results Table IV
Probability of someone using one of the three most popular aliases for an object

Editor-5	Editor-25	Decoder	Common Objects	Class-ifieds	Recipe Key-words	
.37	.42	.38	.42	.45	.58	(low estimate)
.38	.49	.41	.48	.59	.67	(high estimate)

This approach looks promising. Unfortunately, the marginal returns for even more aliases diminishes rapidly. Figure 2 presents an operating characteristic, plotting how many optimally selected aliases are needed to account for a given percentage of the untutored users' attempts. The figure shows the results for the Common Objects data, which is typical of our data sets.

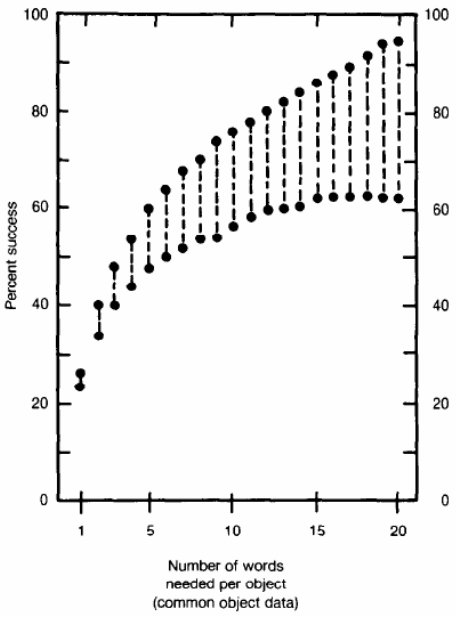

FIGURE 2. Plot of How Many Different Names (or "aliases") Are Needed for Each Object to Account for a Given Percentage of Success (the exact values are not known because of statistical estimation problems, so ranges are indicated by dashed lines)

印刷頁碼 967｜繁體中文譯文

左欄承前

低估計值的取得方式，是用隨機一半的資料挑出最佳詞，再用另一半資料檢驗其效果；高估計值則以資料中最熱門詞的實際相對頻率取得。結果表 II 呈現各資料集這些低估計與高估計的命中率範圍。

Results Table II

結果表 II：某人使用某項目最熱門用語的機率

	Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
低估計	.15	.19	.22	.26	.26	.31
高估計	.16	.22	.21	.28	.34	.36

正文

雖然相較於單純的直覺命名法通常有約 2 比 1 的改善，但仍有 65–85% 的時間會失敗。

由於不會有任何用語比最熱門的用語更常被想到，這些數字代表「每個項目只指派單一存取詞」時的真實效能上限。簡單地說，資料告訴我們：對大多數項目而言，並不存在一個好的存取詞。所謂「顯而易見」、「不言自明」或「自然」的用語，是個迷思！既然連最好的名稱都沒什麼用，那就表示：在「讓不熟悉的使用者也能存取」的意義下，並不存在挑選好名稱的規則、指引或程序。

任何進一步的改善，都需要不同的策略。

3.4 The Value of a Few Aliases → 3.4 少量別名的價值

一旦否定了單一存取詞的想法，下一個合乎邏輯的建議就是提供數個存取詞。例如圖書館目錄與索引，通常會為每個被參照的項目設置至少兩個（但很少超過四個）入口點。

以下呈現一個系統的估計表現：其中每個項目有三個系統已知的存取詞，這些詞以頻率加權隨機抽樣方式選出，模擬一個設計團隊最先想到的三個「直覺提名（armchair nominations）」。

Results Table III

結果表 III：某人使用某項目三個直覺別名之一的機率

Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
.21	.30	.20	.28	.34	.45

雖然相較於單一直覺名稱已有相當的改善，但三個直覺別名並不比一個經最佳挑選的用語更好。

右欄

若改用三個最熱門、也就是三個經最佳挑選的存取詞，可得到以下結果。

Results Table IV

結果表 IV：某人使用某項目三個最熱門別名之一的機率

	Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
低估計	.37	.42	.38	.42	.45	.58
高估計	.38	.49	.41	.48	.59	.67

這種做法看起來很有希望。可惜的是，再增加別名所帶來的邊際效益迅速遞減。圖 2 呈現一條操作特徵曲線，描繪要涵蓋未受指導使用者一定比例的嘗試，需要多少個經最佳挑選的別名。該圖顯示的是 Common Objects 資料的結果，這在我們的資料集中具代表性。

Figure 2（座標軸資訊，標為圖表元素，不得當正文）

〔圖表〕縱軸：Percent success（成功百分比），刻度 0、20、40、60、80、100（左右兩側對稱標示）

橫軸：Number of words needed per object (common object data)（每個項目所需的詞數，常見物件資料），刻度 1、5、10、15、20

圖面：成對圓點以虛線相連，上下兩點分別為高／低統計估計

FIGURE 2 圖說

圖 2. 每個項目需要多少個不同名稱（即「別名」），才能達到給定的成功百分比（由於統計估計上的問題，確切數值未知，因此以虛線標示範圍）

The two curves present the high and low statistical estimates respectively. (Especially for larger numbers of aliases, practical limits are probably closer to the lower than the upper estimate.) Even with fifteen aliases, only 60–80 percent of "attempts" will be satisfied.

Clearly the only hope for untutored vocabulary driven access is to provide many, many alternate entry terms. Thus aliases are, indeed, the answer, but only if used on a much larger scale than usually considered.

The tendency to underestimate the need for so many aliases results, we conjecture, from the fact that any one person (for example, the designer) usually can think of only a handful of the terms that other people would consider appropriate. For the example exercise in Figure 1 at the beginning of the paper it is rare for one person to come up with more than a half dozen names. Across all people, however, over a hundred command names for that service have been suggested, including: activities, calendar, events, cityevents, whatsup, sparetime, funtime, weekender, nightout, downtown, entertain, outings, diversions, play, fun&games, amusements, guide, goingson, happenings, thingstodo, aroundtown, leisure, weekend, nightlife, abouttown, onthetown. . . .

3.5 A Solution is Unlimited Aliasing

Our discussion of the "unfamiliar access" problem has followed the spirit of common practice, and been dominated by a computer-centered point of view. Beginning with the set of system entities, various schemes for assigning access terms were proposed and evaluated. We believe this approach is misleading because it tends to focus on what the computer brings to the interaction (the system's objects), yet it is a characteristic of what the human brings, the variability in vocabulary usage, that dominates the problem.

If we want the system to have the best chance of "understanding" the user, it is better to conceive of design the other way around; begin with user's words, and find system interpretations. We need to know, for every word that users will try in a given task environment, the relative frequencies with which they would be satisfied by various objects. The system would be designed using a table of word usage much like the ones we have collected. When users attempt access with a word, the highest frequency object for that word is the system's best guess as to the user's intent. For example in Table 1(b), when confronted by a user's term "fruit," the best guess is a PEAR. If we had complete data this approach would yield the theoretically optimum performance for getting users and objects together.

In terms of the designer's usual system-oriented standpoint, the proposal here is to allow essentially unlimited numbers of aliases. This is *not* to say that designers should build systems with unruly thousands of commands or objects in them. The number of distinct objects or commands needed is a matter of system functionality and good design.[5] But, regardless of the number of commands or objects in a system and whatever the choice of their "official" names, *the designer must make available many, many alternate verbal access routes to each.*

3.5.1 *The Precision Problem.* Before we discuss the practicality and estimated performance of this approach, we must face what is called, in information retrieval literature, the "precision" problem.

Whenever a given word is applied to more than one object, ambiguity results. For example, in the data of Table Ib, PEAR (frequency 19) is the best guess for "fruit," but the user might also mean NECTARINE (frequency 12) or even RAISIN (frequency 1). The "fruit" example illustrates imprecision arising from the generality of a term. It may also arise from polysemy—the same word may mean two or more distinct things. In a retrieval based on the term "nut" a system can only guess whether to return items related to hardware, seeds or eccentrics.

How serious is the precision problem? In our data the probabilities that two people intended the same referent(s) by a given term were:

Results Table V
Probability that two people using a term intend the same object

Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
.41	.15	.73	.52	.62	.13

Here there was considerable variation between data sets. In the recipe data, two uses of a given keyword refer to the same recipe only about an eighth of the time, reflecting the fact that cooking terms often refer to several recipes. In contrast, two occurrences of a particular name for the hypothetical decoder referred to the same action three quarters of the time, indicating more restricted and precise usage.

There was a modest but reliable tendency within each data set for less frequent terms to be more precise [12]. Often more popular words, while being more likely for a given object, are also more likely for other objects (e.g., "change" in Table 1(a)). This observation is important since it means unlimited aliasing will not incur a great precision cost. Having more access words per object does not logically imply more objects per word, and empirically the average tendency appears to be the reverse. Moreover, in an interactive situation in which users can refine their selections by a series of

[5] In fact, careful system design may itself help the untutorec's vocabulary access problem. For example, if a system's intrinsic structure can be designed to support it, a factorial task decomposition can lead to predictable patterns of official terminology, perhaps allowing users to predict some terminology from others ("delete word," "transpose word," "delete sentence," ⇒ "transpose sentence").

印刷頁碼 968｜繁體中文譯文

左欄

兩條曲線分別代表統計上的高估計與低估計。（尤其在別名數量較多時，實務上限可能更接近下界而非上界。）即使有十五個別名，也只有 60–80% 的「嘗試」能被滿足。

顯然，未受指導的詞彙驅動存取唯一的希望，就是提供非常非常多的替代入口詞。因此別名確實是答案，但前提是其規模要遠大於一般人所設想的。

我們推測，人們之所以低估所需別名的數量，是因為任何單一個人（例如設計者）通常只能想到別人會認為適當的用語中的少數幾個。以本文開頭圖 1 的示範練習為例，一個人很少能想出超過半打的名稱。然而綜合所有人，該服務被提出的命令名稱超過一百個，包括：activities、calendar、events、cityevents、whatsup、sparetime、funtime、weekender、nightout、downtown、entertain、outings、diversions、play、fun&games、amusements、guide、goingson、happenings、thingstodo、aroundtown、leisure、weekend、nightlife、abouttown、onthetown……

3.5 A Solution is Unlimited Aliasing → 3.5 解方是不限量別名

我們對「不熟悉者存取」問題的討論，一直沿著常規做法的思路，並被以電腦為中心的觀點所主導：先從系統實體的集合出發，再提出並評估各種指派存取詞的方案。我們認為這種取徑具有誤導性，因為它偏重電腦帶進互動中的東西（系統的物件），然而真正主導這個問題的，卻是人帶進來的特性——詞彙使用的變異性。

如果我們希望系統有最大的機會「理解」使用者，較好的做法是反過來構思設計：從使用者的詞出發，再去找系統端的解讀。我們需要知道，在特定任務環境中使用者會嘗試的每一個詞，各個項目能滿足他們的相對頻率為何。系統應該用一份很像我們所蒐集的那種用詞頻率表來設計。當使用者用某個詞嘗試存取時，該詞頻率最高的項目就是系統對使用者意圖的最佳猜測。例如在表 I(b) 中，面對使用者輸入的 "fruit"，最佳猜測是
PEAR。若我們擁有完整資料，這種做法將能在撮合使用者與項目上達到理論上的最佳表現。

就設計者慣有的、以系統為導向的立場而言，本文的提案是允許基本上不受限量的別名。這並不是說設計者應該打造內含多到失控的數千個命令或項目的系統。所需的相異項目或命令數量，是系統功能與良好設計的問題。[5] 但是，無論系統中命令或項目的數量多寡，也無論其「官方」名稱如何選定，設計者都必須為每一個提供非常非常多的替代語文存取途徑。

3.5.1 The Precision Problem → 3.5.1 精確率問題

在討論這種做法的可行性與預估表現之前，我們必須先面對資訊檢索文獻中所謂的「精確率（precision）」問題。

只要同一個詞被套用到一個以上的項目，就會產生歧義。例如在表 Ib 的資料中，PEAR（頻次 19）是 "fruit" 的最佳猜測，但使用者也可能是指 NECTARINE（頻次 12），甚至 RAISIN（頻次 1）。"fruit" 這個例子說明的是源自用語過於一般（泛指性）的不精確。不精確也可能源自一詞多義（polysemy）——同一個詞可能表示兩種以上不同的事物。在以 "nut"
一詞進行的檢索中，系統只能猜測該回傳與五金螺帽、堅果種子，還是與怪人有關的項目。

精確率問題有多嚴重？在我們的資料中，兩個人使用同一用語卻指向相同指涉對象的機率如下：

Results Table V

結果表 V：兩個人使用同一用語卻意指同一項目的機率

Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
.41	.15	.73	.52	.62	.13

這裡各資料集之間差異相當大。在食譜資料中，同一個關鍵詞的兩次使用只有大約八分之一的時候指向同一道食譜，這反映了烹飪用語往往同時指涉多道食譜。相對地，那個假想解碼器的同一個名稱出現兩次時，有四分之三的時候指向同一個動作，顯示其用法較受限也較精確。

在每個資料集內部，都存在一種幅度不大但穩定的傾向：較低頻的用語較為精確[12]。較熱門的詞雖然對某個特定項目的可能性較高，但對其他項目的可能性通常也較高（例如表 I(a) 中的 "change"）。這項觀察很重要，因為它意味著不限量別名並不會付出很大的精確率代價。每個項目擁有更多存取詞，在邏輯上並不蘊含每個詞對應更多項目；而就實證而言，平均趨勢似乎正好相反。此外，在使用者可以透過一連串輸入逐步收斂其選擇的互動情境中……

註 5

[5] 事實上，審慎的系統設計本身或許就能減輕未受指導者的詞彙存取問題。例如，若系統的內在結構能支援，採用因子式的任務分解可以產生可預測的官方術語型態，也許能讓使用者從已知術語推測其他術語（"delete word"、"transpose word"、"delete sentence" ⇒ "transpose sentence"）。

inputs, the higher hit rates afforded by extensive aliasing has been found to be extremely beneficial even when it does carry with it a lower average precision [5].

3.5.2 *Dealing with Imprecision: Disambiguation.* Our data show that any keyword system capable of providing a high hit rate for unfamiliar users must let them use words of their own choice for objects. But, because of the inherent amgibuity of users' own words, the system can never be sure it has correctly inferred the user's referent; it can only make good guesses. The cost of an incorrect guess must therefore be considered. While users might be happy with a system's mere guess about a book to look at, many commands are too consequential to be executed on guessed intent (e.g., "delete file") and so the inherent vagueness in words must be resolved.

There are two approaches: either make the user memorize precise system meanings, or have the user and system interact to identify the precise referent. The latter might possibly be done in many ways, including interpretation of multiterm Boolean expressions[6], formal query languages or "natural language" understanding, although none of these has demonstrated notable success to date. A simpler approach, which our data let us model, is to return a set of choices to the user whenever there is ambiguity. If we assume that in the restricted context of a particular application the user can recognize the correct interpretation on sight, there is probably no faster route to resolution. As we will show below, with unlimited aliases the number of alternatives to be returned for user selection will usually be small. Alternatives can be ordered by frequency, the most likely objects first. Nevertheless, correct recognition by the user requires adequate description of the meaning or content of each choice, perhaps including more precise terms, examples, etc. It must be realized that this too is a highly error prone communication process [2].

One question raised by the unlimited alias proposal is whether people will acquire undesirable habits if the system is so tolerant. We think this problem is self limiting. As mentioned, in command execution (though not necessarily in information retrieval) spontaneous entries will often require disambiguation procedures. Since such procedures are inherently costly to users, there is incentive for them to move toward more efficient language. This suggests using the unlimited alias system only as an index into a help facility. When users invoke it with their own words, the system would pick its best guesses, and present them in a menu. Each guess would be labeled by some "standard" access terminology and be accompanied by a description of the standard referent. Actual execution would always be via the standard "name," thereby encouraging the learning of precise terms required to take the disambiguation process out of the loop.

3.5.3 *Performance of Unlimited Aliasing Systems.* We wish to know how well unlimited aliasing can work in principle and what can be expected in practice.

First, how good would performance be given an infinite amount of data? By definition the system would recognize all user words. However, it might have to make several guesses about the intended object. Assume the system asked the user to verify one object at a time, and the user was only satisfied by exactly one object in the database. Then, for our data sets, the median number of guesses before success in each disambiguation dialogue would be approximately as follows (the number in parentheses is a reminder of how many objects there were in the domain and is useful to help judge the magnitude of the accomplishment).

Results Table VI
Median number of system object guesses for success

	Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
Guesses	1	3	1	1	1	4
Objects in domain	(5)	(25)	(12)	(50)	(64)	(188)

In real life, we can never have complete data on word-object usage. Incomplete data can have several detrimental effects, the most important being that the system might not know a user's word at all, or only have it listed with wrong objects. This would lead to a complete miss, of the sort found very frequently in armchair naming methods.

The seriousness of this problem varied from domain to domain. In the decoder data, even after 100 subjects were asked about each object, a new person would have had a 28 percent chance of proposing a completely new term. In contrast, after only 24 people had proposed keywords for a recipe, there was less than one chance in 50 that the next person would come up with a new term.

There is no single number characterizing how overall performance would suffer from incomplete data; it would depend on how much data was collected and the variability of naming in the given domain. We estimated how well a system could do in our domains with half the amount of data we originally collected. At the median number of guesses needed with infinite data, performance was degraded by approximately one third when restricted to half our available data: performance was cut in half for Classifieds using only 8 subjects'

[6] Note that in terms of a system correctly recognizing untutored users' terms, Boolean conjunctions *multiply* the single-term problem addressed in this paper. For example, indexer and retriever might agree to call large things, "big" with probability ~0.1, and circular things, "round," with probability ~0.2. Then, assuming independence, the Boolean combination, "big and round" will retrieve only 0.02 of the desired large, circular things.

印刷頁碼 969｜繁體中文譯文

左欄承前

……（承前）已發現廣泛使用別名所帶來的較高命中率極為有益，即使它同時伴隨較低的平均精確率 [5]。

3.5.2 Dealing with Imprecision: Disambiguation → 3.5.2 處理不精確：消歧義

我們的資料顯示，任何想為不熟悉的使用者提供高命中率的關鍵詞系統，都必須讓他們用自己選擇的詞來指稱項目。但由於使用者自身用詞本有的歧義，系統永遠無法確定自己正確推斷出了使用者的指涉對象；它只能做出好的猜測。因此必須考慮猜錯的代價。使用者或許樂於接受系統對「該看哪本書」的單純猜測，但許多命令的後果太過嚴重，不能只憑猜測的意圖就執行（例如 "delete file"），因此詞語本有的含混必須被消解。

有兩種做法：要嘛讓使用者背下精確的系統語意，要嘛讓使用者與系統互動以確認精確的指涉對象。後者可能有許多實作方式，包括解讀多詞布林運算式[6]、形式查詢語言，或「自然語言」理解——不過這些至今都未展現出顯著的成效。有一種較簡單的做法是我們的資料能夠建模的：只要出現歧義，就回傳一組選項給使用者。如果我們假設在特定應用的受限脈絡下，使用者一看就能認出正確的解讀，那大概沒有更快的解決途徑了。如下文所示，在不限量別名之下，需要回傳給使用者挑選的候選項通常很少。候選項可依頻率排序，最可能的項目排在最前面。儘管如此，要讓使用者正確辨認，仍需對每個選項的意義或內容給出充分的描述，或許包含更精確的用語、範例等。必須認清，這同樣是一個極易出錯的溝通過程 [2]。

不限量別名的提案引出一個問題：若系統如此寬容，人們會不會養成不良習慣？我們認為這個問題會自我限制。如前所述，在命令執行時（資訊檢索則未必如此），自發輸入往往需要走消歧義流程。由於這類流程對使用者本身就有成本，他們便有誘因轉向更有效率的用語。這也暗示：不限量別名系統只宜當作進入輔助說明機制的索引。當使用者用自己的詞喚起它時，系統會挑出最佳猜測並以選單呈現；每個猜測都標上某個「標準」存取用語，並附上該標準指涉對象的描述。實際執行則一律透過標準「名稱」，藉此鼓勵使用者學會所需的精確用語，讓消歧義流程最終得以退出迴圈。

註 6

[6] 請注意，就系統能否正確辨識未受指導使用者的用語而言，布林合取（AND）會把本文所處理的單詞問題相乘。舉例來說，索引員與檢索者可能對「大的東西」都叫 "big"，機率約 0.1；對「圓的東西」都叫 "round"，機率約 0.2。那麼在獨立性假設下，布林組合 "big and round" 只能檢索到所欲之「又大又圓」物件的 0.02。

3.5.3 Performance of Unlimited Aliasing Systems → 3.5.3 不限量別名系統的表現

我們想知道不限量別名在原則上能運作得多好，以及在實務上可以期待什麼。

首先，若擁有無限量的資料，表現會有多好？依定義，系統會辨識所有使用者的詞。不過它可能得對意圖的項目做出好幾次猜測。假設系統每次請使用者確認一個項目，而資料庫中恰好只有一個項目能滿足使用者。那麼對我們的資料集而言，每次消歧義對話中成功前所需的猜測次數中位數大致如下（括號中的數字提醒該領域共有多少項目，有助於判斷這項成果的量級）。

Results Table VI

結果表 VI：成功前系統猜測項目次數的中位數

	Editor-5	Editor-25	Decoder	Common Objects	Classifieds	Recipe Keywords
猜測次數 Guesses	1	3	1	1	1	4
領域項目數 Objects in domain	(5)	(25)	(12)	(50)	(64)	(188)

右欄

在現實中，我們永遠不可能擁有詞—項目使用情形的完整資料。資料不完整會造成幾種不利影響，其中最重要的是：系統可能根本不認得使用者的詞，或只把它列在錯誤的項目底下。這會導致完全落空，正是直覺命名法中極常見的那種失敗。

這個問題的嚴重程度因領域而異。在解碼器資料中，即使每個項目都已問過 100 位受試者，下一位新受試者仍有 28% 的機率提出一個全新的用語。相對地，食譜關鍵詞只要問過 24 個人，下一個人提出新用語的機率就不到五十分之一。

沒有單一數字能刻畫整體表現會因資料不完整而受損多少；那取決於蒐集了多少資料，以及該領域命名的變異性。我們估計了在只用原先所蒐集資料一半的情況下，系統在各領域能有多好的表現。在「無限資料下所需猜測次數的中位數」這個標準上，限制為一半可用資料時表現約下降三分之一：Classifieds 只用 8 位受試者的資料時表現腰斬……

data, cut by a third for the Common Objects using 178 subjects' data, and cut by only 10 percent for the Editor-5 data using 24 subjects' data.

4. Summary and Discussion

We have been studying vocabulary problems in using unfamiliar computer applications. We can summarize the findings in a few major points. First, a single access term (e.g., a "name") chosen by a single designer will provide very poor access (10–20 percent hit rates). Second, a single empirically optimized term is much better; and will work as well as 2–3 armchair aliases together. Third, to achieve really good performance, very many aliases are needed. An empirically based, frequency weighted "unlimited aliases" system can often produce 50 to 100 percent hit rates in its first three guesses to untutored queries, depending on the domain and the amount of data collected. In the limit, the only barrier is precision, and our data show that frequency data on the possible objects intended by a given term will usually provide good ambiguity resolution.

This brings us to the conclusion that reasonable access can be provided for the untutored, but only if an extensive table of word usage behavior is compiled. Such a solution is technically simple, but potentially tedious. We note that to some extent the collection of multiple aliases will arise in any well done iterative interface design. Good, et al. [6] provide such an example. They report that alias collection accounted for the largest share of the dramatic interface performance improvements resulting from a careful iterative design process with extensive user testing.

Iterative design of interfaces is very useful for many reasons, but our analyses suggest the vocabulary problem can be attacked separately. But what can be done about the tedium and cost? Fortunately there appear to be several attractive alternatives. A "quick and dirty" approach is to get a fair number, say 4 to 8 representative users to supply a fair number, say 3 to 6, terms apiece for each object. An experiment by Gomez and Lochbaum [5] with a small interactive search system obtained the predicted four-fold increase in users' ability to find a desired recipe using this approach. This technique is most likely to be cost-effective for databases of moderate size that are expected to remain stable over long periods and be used infrequently by many people.

A more familiar approach for textual objects is to extract multiple access terms automatically from the content. The recipe texts contain roughly 80 percent of the keywords selected by users, and in an experiment to be reported elsewhere, Gomez, Lochbaum and Landauer found full-text indexing to be roughly equivalent to extensive alias "harvesting" from experts for the recipe database.

A third, exciting possibility is to construct unlimited alias indices adaptively, on site, in use. Such adaptive indexing methods are discussed elsewhere [3] but the idea is simple and has some partial precursors in the literature [9] [10]. Basically an adaptive indexing system begins with an index obtained in an ordinary way (for example, by armchair naming). Users try their own words to access desired objects, and, on their first few tries, usually fail. But sometimes they will eventually find something they want. With user concurrence, the initially unsuccessful words are added to the system's usage table. The next time someone seeks the object, the index is more knowledgeable. Adaptive indexes collect the needed data relatively painlessly, from the right users under authentic conditions. They also start to pay off rapidly, since they automatically tend to focus data collection on the most sought-after objects. Trials of such systems show that they produce the kind of large performance improvements predicted in this paper for systems with massive alias lists.

Thus we can augment our earlier conclusion to say that not only is drastic improvement needed and possible, but also practicable.

In closing, we would like to comment briefly on the research strategy that was employed in this work. Typically, human-computer interaction research has involved the comparison of two or more particular systems or features, either by model-based analysis (e.g., Card, Moran and Newell [1]), by a one-time "contest" (e.g., Roberts and Moran [11]), or by successive iterative test (e.g., Good, et al. [6]). This approach is valuable in increasing the speed and accuracy with which the field can choose good examples to follow. But, because of the complexity of most systems, it is limited in the generality of its conclusions.

By contrast, the approach exemplified here begins with what we call a failure analysis. In particular, we look for failures of humans to obtain desired results in interactions with computers. In the present case, we found that people often enter words that fail to be correctly recognized by systems. We then collect experimental or observational data, do simulations and model building — whatever is necessary to understand what the human can and will do in the situation, and what is needed on the part of the machine in order that the combination of human and machine can succeed more often. In the present case, we discovered unexpectedly large and pervasive variability in spontaneous term application, and learned that systems need to recognize a very rich variety of aliases. If the emerging machine requirements can be realized practically, the next step to invention or improved design is straightforward. In the present case, many avenues for interface improvements are apparent.

It is essential to stress that in this approach inventions and design improvements come from basic understanding of performance problems and human behavior, not from analysis or tests of alternative designs. The extreme variability of word selection by humans is a fundamental fact of human behavior found by studying a performance problem. Knowledge of this fact can lead to better system design in a wide range of applications, not just in the choice of one system over another.

印刷頁碼 970｜繁體中文譯文

左欄承前

……Common Objects 只用 178 位受試者的資料時下降三分之一，而 Editor-5 只用 24 位受試者的資料時僅下降 10%。

4. Summary and Discussion → 4. 總結與討論

我們一直在研究使用不熟悉的電腦應用時的詞彙問題。研究發現可以歸納為幾個要點。第一，由單一設計者選定的單一存取詞（例如一個「名稱」）所提供的存取能力非常差（命中率 10–20%）。第二，單一經實證最佳化的用語好得多，其效果約相當於 2–3 個直覺別名合起來。第三，要達到真正良好的表現，需要非常多的別名。一套以實證為基礎、依頻率加權的「不限量別名」系統，對未受指導的查詢，往往能在前三次猜測內達到 50% 到 100% 的命中率，實際數值取決於領域與所蒐集的資料量。到了極限，唯一的障礙是精確率；而我們的資料顯示，關於某個用語可能意指哪些項目的頻率資料，通常足以提供良好的歧義消解。

這使我們得到一個結論：確實可以為未受指導者提供合理的存取，但前提是必須編纂一份廣泛的用詞行為表。這樣的解法在技術上很簡單，卻可能相當繁瑣。我們注意到，在任何做得好的迭代式介面設計中，某種程度上本來就會蒐集到多個別名。Good 等人 [6] 就是一個例子：他們指出，在一個搭配大量使用者測試的審慎迭代設計流程中，介面效能的顯著改善有最大一部分來自別名的蒐集。

迭代式介面設計因為許多理由而非常有用，但我們的分析顯示，詞彙問題可以單獨處理。然而繁瑣與成本該怎麼辦？所幸似乎有幾個有吸引力的替代方案。一個「快而粗糙（quick and dirty）」的做法，是找相當數量（比方說 4 到 8 位）的代表性使用者，請每人為每個項目各提供相當數量（比方說 3 到 6 個）的用語。Gomez 與 Lochbaum [5] 以一套小型互動檢索系統所做的實驗，用這個方法得到了預測中的四倍提升——使用者找到目標食譜的能力提高了四倍。這項技術最可能具成本效益的場合，是規模中等、預期長期保持穩定、且被許多人偶爾使用的資料庫。

對文字型項目而言，更為人熟悉的做法是從內容中自動抽取多個存取詞。食譜文本本身就含有使用者所選關鍵詞的大約 80%；在另文報告的一項實驗中，Gomez、Lochbaum 與 Landauer 發現，就食譜資料庫而言，全文索引的效果大致相當於向專家大量「採集（harvesting）」別名。

右欄

第三種令人振奮的可能性，是在現場、在使用過程中自適應地建構不限量別名索引。這類自適應索引方法在別處已有討論 [3]，其構想很簡單，而且在文獻中已有部分先例 [9][10]。基本上，自適應索引系統一開始使用以尋常方式（例如直覺命名）取得的索引。使用者用自己的詞去存取想要的項目，前幾次通常會失敗；但有時候他們最終會找到想要的東西。在使用者同意下，那些一開始不成功的詞會被加入系統的使用表。下一次有人尋找同一個項目時，索引就更「懂」了。自適應索引能相對無痛地蒐集所需資料，而且來自真正的使用者、在真實條件下產生。它們也很快就開始有回報，因為它們會自動把資料蒐集集中在最常被尋找的項目上。這類系統的實測顯示，它們確實產生了本文對擁有大量別名清單的系統所預測的那種大幅效能改善。

因此我們可以強化先前的結論：大幅改善不只是必要且可能的，而且是可行的。

最後，我們想簡短談談這項工作所採用的研究策略。人機互動研究通常是比較兩個以上的特定系統或功能，方式包括以模型為基礎的分析（例如 Card、Moran 與 Newell

[1]）、一次性的「比賽（contest）」（例如 Roberts 與 Moran [11]），或連續的迭代測試（例如 Good 等人 [6]）。這種取徑在提高該領域挑選良好典範的速度與準確度上很有價值。但由於多數系統的複雜性，其結論的普遍性有限。

相對地，本文所示範的取徑始於我們所謂的失效分析（failure analysis）。具體而言，我們尋找人類在與電腦互動時未能取得預期結果的失效情形。在本例中，我們發現人們經常輸入系統無法正確辨識的詞。接著我們蒐集實驗或觀察資料、進行模擬與建模——凡是有助於理解人在該情境下能做什麼、會做什麼，以及機器端需要具備什麼才能讓人機組合更常成功的工作，我們都做。在本例中，我們發現自發用語的變異性大得出乎意料且無所不在，並學到系統需要辨識極為豐富多樣的別名。一旦浮現的機器需求能在實務上實現，接下來走向發明或改良設計就水到渠成。在本例中，介面改良的許多途徑都已清楚可見。

必須強調的是，在這種取徑下，發明與設計改良來自對效能問題與人類行為的基本理解，而不是來自對替代設計的分析或測試。人類選詞的極端變異性，是透過研究一個效能問題而發現的人類行為基本事實。認識這項事實，能在廣泛的應用中帶來更好的系統設計，而不只是幫我們在兩個系統之間二選一。

SUMMARY: *Many, many alternative access words are needed for users to get what they want from large and complex systems.*

REFERENCES

1. Card, S., Moran, T.P., and Newell, A. *The Psychology of Human-Computer Interaction.* Lawrence Erlbaum Associates, Hillsdale, N.J., 1983.
2. Dumais, S.T., and Landauer, T.K. Describing categories of objects for menu retrieval systems. *Behavior Research Methods, Instruments, & Computers*, 16, 2 (Apr. 1984), 242–248.
3. Furnas, G.W. Experience with an adaptive indexing scheme. *Human Factors in Computer Systems, CHI '85 Proceedings.* Conference held in San Francisco, CA, April 15–18, 1985, 131–135.
4. Furnas, G.W., Landauer, T.K., Gomez, L.M., and Dumais, S.T. Statistical semantics: Analysis of the potential performance of key-word information systems. *Bell System Technical Journal*, 62, 6 (Jul.–Aug. 1983), 1753–1806.
5. Gomez, L.M., and Lochbaum, C.C. People can retrieve more objects with enriched key-word vocabularies. But is there a human performance cost? In B. Shackel (Ed.) *Human-Computer Interaction—Interact '84.* North-Holland, Amsterdam, 257–261.
6. Good, M.D., Whiteside, J.A., Wixon, D. R., and Jones, S.J. Building a user-derived interface. *Commun. ACM*, 27, 10 (Oct. 1984), 1032–1043.
7. Herdan, G. *Type Token Mathematics: A Textbook of Mathematical Linguistics*, S-Gravenhage, Mouton, 1960.
8. Landauer, T.K., Galotti, K., and Hartwell, S. Natural command names and initial learning: A study of text editing terms. *Commun. ACM*, 26, 7 (Jul. 1983), 495–503.
9. Reisner, P. Construction of a growing thesaurus by conversational interaction in a man-machine system. *Proceedings of the American Documentation Institute*, 26th Annual Meeting, Chicago, Ill. October 1963.
10. Reisner, P. Evaluation of a 'Growing Thesaurus'. Research Paper RC-1662, August 9, 1966, IBM Watson Research Center, Yorktown Heights, N.Y.
11. Roberts, T.L., and Moran, T.P. The evaluation of text editors: Methodology and empirical results. *Commun. ACM*, 26, 4 (Apr. 1983), 265–283.
12. Sparck-Jones, K. A Statistical interpretation of term specificity and its application in retrieval. *J. Doc.* 28, 1 (Mar. 1972), 11–21.
13. Whalen, T., and Latremouille, S. The effectiveness of a tree-structured index when the existence of information is uncertain. *Teledon Behavioral Research 2: The Design of Videotex Tree Indices*, Ottawa, Canada: Department of Communications, (May 1981), pp. 3–12.
14. Zipf, G.K. *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology.* Addison-Wesley, Reading, Mass., 1949.

CR Categories and Subject Descriptors: H.1.2 [**Models and Principles**]: User/Machine Systems—*human factors, human information processing*; H.3.1 [**Information Storage and Retrieval**]: Content Analysis and Indexing—*Indexing Methods, Linguistic Processing, Thesauruses*

General Terms: Human Factors

Additional Key Words and Phrases: human–computer communication, human–computer interaction, keyword, repeat rate, vocabulary, Zipf law

Received 7/85; revised 4/86; accepted 12/86

Authors' Present Address: G.W. Furnas, T.K. Landauer, L.M. Gomez, S.T. Dumais, Bell Communications Research, Inc., 435 South Street, Morristown, NJ 07960.

印刷頁碼 971｜繁體中文譯文

SUMMARY（頁首斜體標語）

摘要： 使用者要從龐大而複雜的系統中取得他們想要的東西，需要非常非常多的替代存取詞。

REFERENCES（依委託：只校正書目文字，不譯作者姓名與刊名）

原文照錄之正確書目：

1. Card, S., Moran, T.P., and Newell, A. The Psychology of Human-Computer Interaction. Lawrence Erlbaum Associates, Hillsdale, N.J., 1983.

2. Dumais, S.T., and Landauer, T.K. Describing categories of objects for menu retrieval systems. Behavior Research Methods, Instruments, & Computers, 16, 2 (Apr. 1984), 242–248.

3. Furnas, G.W. Experience with an adaptive indexing scheme. Human Factors in Computer Systems, CHI '85 Proceedings. Conference held in San Francisco, CA, April 15–18, 1985, 131–135.

4. Furnas, G.W., Landauer, T.K., Gomez, L.M., and Dumais, S.T. Statistical semantics: Analysis of the potential performance of key-word information systems. Bell System Technical Journal, 62, 6 (Jul.–Aug. 1983), 1753–1806.

5. Gomez, L.M., and Lochbaum, C.C. People can retrieve more objects with enriched key-word vocabularies. But is there a human performance cost? In B. Shackel (Ed.) Human-Computer Interaction—Interact '84, North-Holland, Amsterdam, 257–261.

6. Good, M.D., Whiteside, J.A., Wixon, D. R., and Jones, S.J. Building a user-derived interface. Commun. ACM, 27, 10 (Oct. 1984), 1032–1043.

7. Herdan, G. Type Token Mathematics: A Textbook of Mathematical Linguistics, S-Gravenhage, Mouton, 1960.

8. Landauer, T.K., Galotti, K., and Hartwell, S. Natural command names and initial learning: A study of text editing terms. Commun. ACM, 26, 7 (Jul. 1983), 495–503.

9. Reisner, P. Construction of a growing thesaurus by conversational interaction in a man-machine system. Proceedings of the American Documentation Institute, 26th Annual Meeting, Chicago, Ill. October 1963.

10. Reisner, P. Evaluation of a 'Growing Thesaurus'. Research Paper RC-1662, August 9, 1966, IBM Watson Research Center, Yorktown Heights, N.Y.

11. Roberts, T.L., and Moran, T.P. The evaluation of text editors: Methodology and empirical results. Commun. ACM, 26, 4 (Apr. 1983), 265–283.

12. Sparck-Jones, K. A Statistical interpretation of term specificity and its application in retrieval. J. Doc. 28, 1 (Mar. 1972), 11–21.

13. Whalen, T., and Latremouille, S. The effectiveness of a tree-structured index when the existence of information is uncertain. Teledon Behavioral Research 2: The Design of Videotex Tree Indices, Ottawa, Canada: Department of Communications, (May 1981), pp. 3–12.

14. Zipf, G.K. Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology. Addison-Wesley, Reading, Mass., 1949.

統一術語表

以下術語已依 Claude 第二核心複核統一，正文亦採相同用法。

英文	統一中文	使用備註／
vocabulary problem	詞彙問題	—
access / to access	存取	全文統一使用此譯法
access term	存取詞	全文統一使用此譯法
entry point / entry term	入口點／入口詞	—
object	物件／項目	全文擇一；本包 README 用「項目」，建議正文統一「項目」，僅在原文並列 object/item 時分別譯「物件」「項目」
item	項目	—
term / word / descriptor / name	用語／詞／描述詞／名稱	原文四詞並列，須分別對應
Armchair naming / armchair method	直覺命名（Armchair）／直覺命名法	全文統一使用此譯法
armchair nominations	直覺提名	—
alias / aliasing	別名／別名化	全文統一使用此譯法
unlimited aliasing	不限量別名	首次出現附英文
untutored (user)	未受指導（的使用者）	全文統一使用此譯法
untrained	未受訓練	—
heavy user	重度使用者	全文統一使用此譯法
subject(s)	受試者	全文統一使用此譯法
hit rate	命中率	全文統一使用此譯法
first-try success	初次嘗試即成功	—
recall	召回率	—
precision	精確率	全文統一使用此譯法

英文	統一中文	使用備註／
ambiguity	歧義	—
disambiguation / disambiguation procedure	消歧義／消歧義流程	全文統一使用此譯法
ambiguity resolution	歧義消解	全文統一使用此譯法
referent	指涉對象	全文統一使用此譯法
polysemy	一詞多義	全文統一使用此譯法
generality (of a term)	（用語的）一般性／泛指性	全文統一使用此譯法
superordinate category	上位類別	全文統一使用此譯法
repeat rate	重複率	全文統一使用此譯法
Zipf's distribution / Zipf law	齊夫分布／Zipf 定律	全文統一使用此譯法
indexer / searcher / retriever	索引員／檢索者	—
indexing / index	索引（動詞：建索引）	全文統一使用此譯法
adaptive indexing	自適應索引	—
full-text indexing	全文索引	—
alias "harvesting"	別名「採集」	—
elicitation technique	誘出技術	—
operating characteristic	操作特徵（曲線）	—
item-centered / word-centered	以項目為中心／以詞為中心	—
failure analysis	失效分析	—
human factors	人因	全文統一使用此譯法
iterative design	迭代式設計	—

英文	統一中文	使用備註／
Boolean conjunction / expression	布林合取／布林運算式	—
single-term problem	單詞問題	全文統一使用此譯法
tedium	繁瑣	—
a fair number	相當數量	全文統一使用此譯法
by factors of three to five	三到五倍	全文統一使用此譯法
thesaurus	詞庫／索引典	—
dialog（natural language dialog）	對話	全文統一使用此譯法
paper（this paper）	本文	全文統一使用此譯法
in the present case	在本研究中／在本例中	全文統一使用此譯法
Association for Computing Machinery	美國計算機協會（ACM）	—
資料集名（Editor-5 / Editor-25 / Decoder / Common Objects / Classifieds / Recipe Keywords）	一律保留英文	不得譯為「慣用類」「金鑰」等